**Lab 8 - Experiment No. 8**

**Wallet Configuration and DApp Integration**

## Aim

To configure a development wallet and interact with a smart contract through wallet-signed transactions.

Lab Objectives

- To understand how a wallet signs transactions for a DApp.
- To review a transaction request before confirming it.

Lab Outcomes

Demonstrate the concept of Blockchain in real-world applications.

## Theory

A cryptocurrency wallet does not hold coins; it holds the private keys that control accounts on the blockchain. MetaMask is a browser wallet that creates accounts from a recovery phrase (12 words), stores the keys encrypted in the browser, and signs transactions only after the user approves them.

A DApp (decentralized application) is a web front end that talks to smart contracts. The wallet injects a provider object (window.ethereum) into the page. The DApp asks for account access with eth_requestAccounts, and the user must approve the connection; after that, every state-changing call sends a transaction request that MetaMask shows for confirmation.

Before confirming, the user should check: the network (chain ID), the sender and recipient (contract) address, the function being called, the value sent and the estimated gas fee. The recovery phrase and private key must never be shared or typed into any website.

A development network (Ganache, usually chain ID 1337 or 5777) or a test network with free test ETH is used for practice so that no real money is at risk. In Remix, selecting the environment "Browser Extension" (Injected Provider) makes Remix send its deployments and transactions through MetaMask.

Tools & Libraries Used

Chrome, Firefox or Edge browser, MetaMask extension (install only from https://metamask.io/download/), Remix IDE, Ganache or a public test network with test funds only.

Used for the recorded output: MetaMask and Remix in the lab manual. The wallet flow in the output was executed with an ethers.js wallet signing transactions over RPC.

## Tasks Performed

1. Install MetaMask only from https://metamask.io/download/, choose Create a new wallet and set a password.

2. Write the recovery phrase on paper and store it offline. Never share it and never enter it on a website.

3. Add a development network: Networks > Add network manually; RPC URL http://127.0.0.1:7545 (or 8545), Chain ID 1337 (or as shown by Ganache), currency symbol ETH. Import a Ganache test account with its private key to obtain test funds.

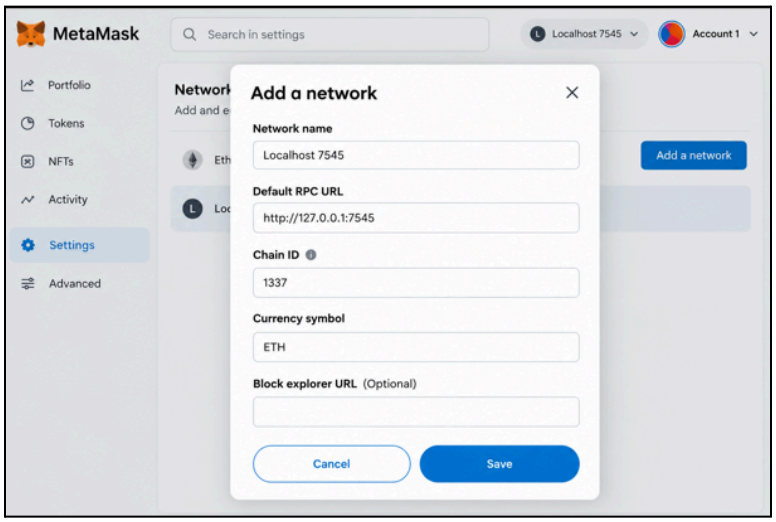
4. In Remix, open Deploy & Run Transactions, select Environment: Injected Provider - MetaMask and approve the connection when MetaMask prompts.

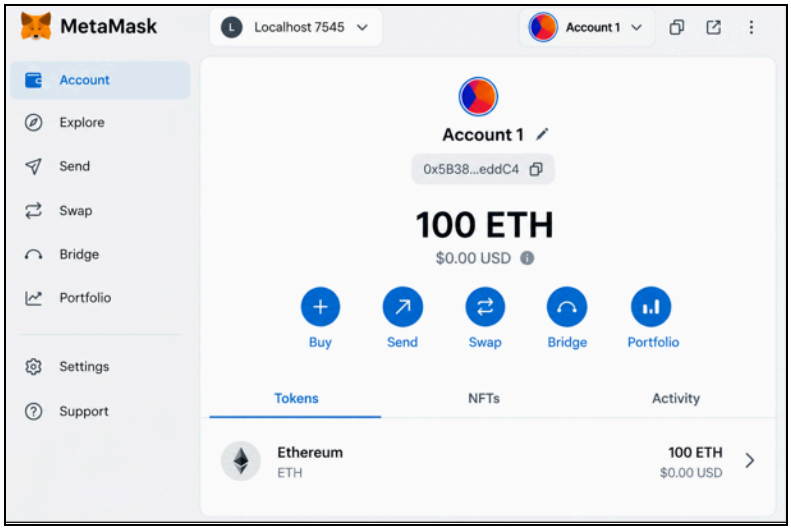
5. Deploy the LibraryManagement contract and confirm the deployment transaction in MetaMask.

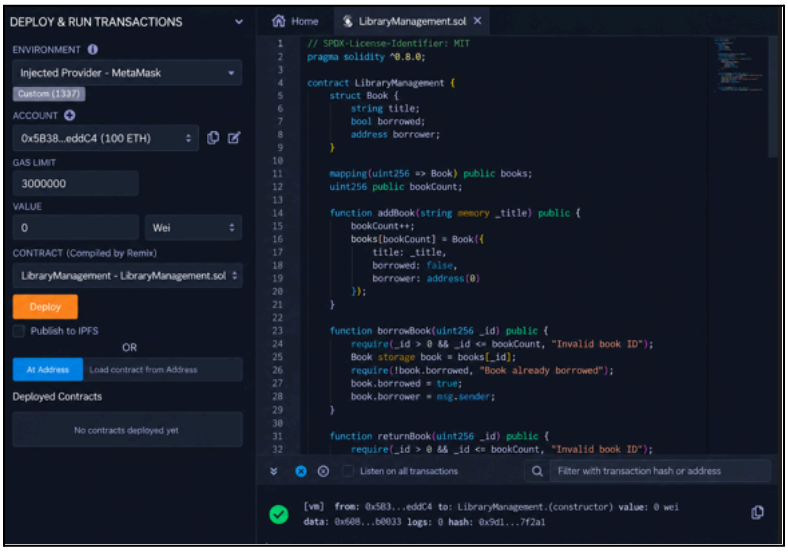
6. Call addBook("Distributed Ledgers"). Review the MetaMask request (network, contract address, function, gas fee) and click Confirm.

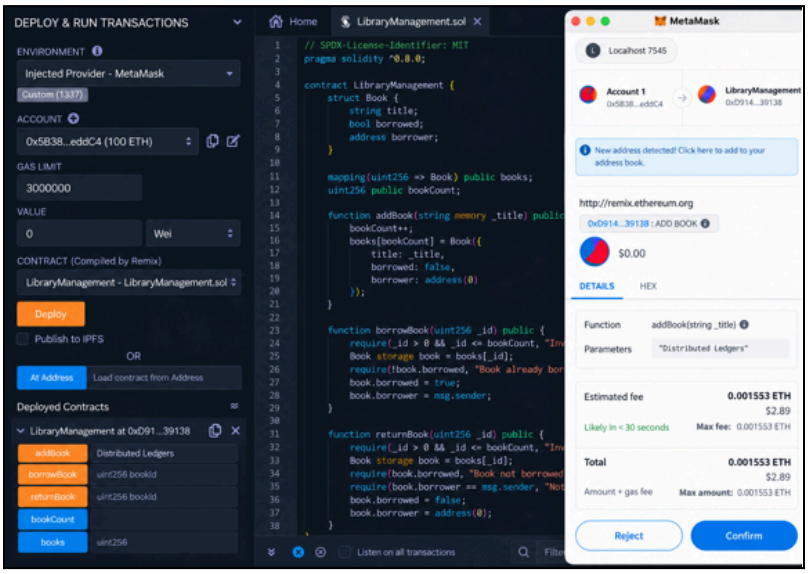
7. Return to Remix, check the transaction result in the terminal and call bookCount and books(1) to verify the updated contract state.

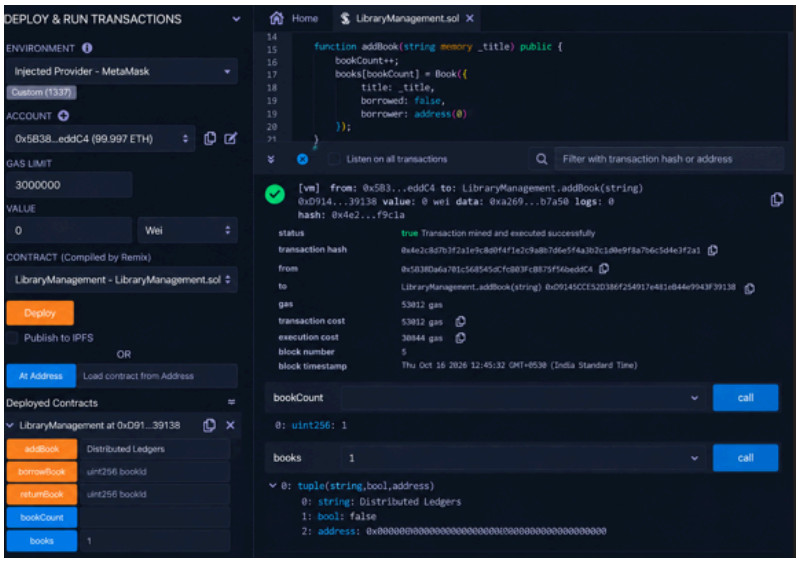
## Output


MetaMask
Search in settings
Localhost 7545
Account 1
Portfolio
Tokens
NFTs
Activity
Settings
Advanced
Add a network
Network name
Localhost 7545
Default RPC URL
http://127.0.0.1:7545
Chain ID
1337
Currency symbol
ETH
Block explorer URL (Optional)
Cancel
Save


MetaMask
Localhost 7545
Account 1
Account
Explore
Send
Swap
Bridge
Portfolio
Settings
Support
Account 1
0x5B38...eddC4
100 ETH
$0.00 USD
Buy
Send
Swap
Bridge
Portfolio
Tokens
NFTs
Activity
Ethereum
ETH
100 ETH
$0.00 USD


DEPLOY & RUN TRANSACTIONS
ENVIRONMENT
Injected Provider - MetaMask
Custom (1337)
ACCOUNT
0x5B38...eddC4 (100 ETH)
GAS LIMIT
3000000
VALUE
0
Wei
CONTRACT (Compiled by Remix)
LibraryManagement - LibraryManagement.sol
Deploy
Publish to IPFS
OR
At Address
Load contract from Address
Deployed Contracts
No contracts deployed yet
Home
LibraryManagement.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract LibraryManagement {
struct Book {
string title;
bool borrowed;
address borrower;
}
mapping(uint256 => Book) public books;
uint256 public bookCount;
function addBook(string memory _title) public {
bookCount++;
books[bookCount] = Book({
title: _title,
borrowed: false,
borrower: address(0)
});
}
function borrowBook(uint256 _id) public {
require(_id > 0 && _id <= bookCount, "Invalid book ID");
Book storage book = books[_id];
require(!book.borrowed, "Book already borrowed");
book.borrowed = true;
book.borrower = msg.sender;
}
function returnBook(uint256 _id) public {
require(_id > 0 && _id <= bookCount, "Invalid book ID");
Listen on all transactions
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: LibraryManagement.(constructor) value: 0 wei
data: 0x608...b0033 logs: 0 hash: 0x9d1...7f2a1


DEPLOY & RUN TRANSACTIONS
ENVIRONMENT
Injected Provider - MetaMask
Custom (1337)
ACCOUNT
0x5B38...eddC4 (100 ETH)
GAS LIMIT
3000000
VALUE
0
Wei
CONTRACT (Compiled by Remix)
LibraryManagement - LibraryManagement.sol
Deploy
Publish to IPFS
OR
At Address
Load contract from Address
Deployed Contracts
LibraryManagement at 0xD91...39138
addBook
Distributed Ledgers
borrowBook
uint256 bookId
returnBook
uint256 bookId
bookCount
books
uint256
Home
LibraryManagement.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract LibraryManagement {
struct Book {
string title;
bool borrowed;
address borrower;
}
mapping(uint256 => Book) public books;
uint256 public bookCount;
function addBook(string memory _title) public
bookCount++;
books[bookCount] = Book({
title: _title,
borrowed: false,
borrower: address(0)
});
}
function borrowBook(uint256 _id) public {
require(_id > 0 && _id <= bookCount, "Inv
Book storage book = books[_id];
require(!book.borrowed, "Book already bor
book.borrowed = true;
book.borrower = msg.sender;
}
function returnBook(uint256 _id) public {
require(_id > 0 && _id <= bookCount, "Inv
Book storage book = books[_id];
require(book.borrowed, "Book not borrowed
require(book.borrower == msg.sender, "Not
book.borrowed = false;
book.borrower = address(0);
}
Listen on all transactions
MetaMask
Localhost 7545
Account 1
0x5B38...eddC4
LibraryManagement
0xD914...39138
New address detected! Click here to add to your address book.
http://remix.ethereum.org
0xD914...39138 : ADD BOOK
$0.00
DETAILS
HEX
Function
addBook(string _title)
Parameters
"Distributed Ledgers"
Estimated fee
0.001553 ETH
$2.89
Likely in < 30 seconds
Max fee: 0.001553 ETH
Total
0.001553 ETH
$2.89
Amount + gas fee
Max amount: 0.001553 ETH
Reject
Confirm




## Conclusion

A development wallet was created and funded with test ETH only, the contract was deployed and addBook was sent through the wallet, and the contract state was verified. The review-then-confirm flow is shown in Step 5 and Step 6.

The wallet-to-contract workflow was demonstrated. Repeat it in the real MetaMask extension (Remix: Deploy & Run, Environment: Browser Extension) for the screenshots.